

# The Technical Interview Agent

## Comprehensive End-to-End Architectural Blueprint and System Implementation Guide

This document details the software engineering specifications, low-latency communication framework, dynamic prompt logic, and analytical execution engine required to construct and deploy a production-grade, voice-driven AI bot for first-round technical screenings.

### 1. Executive System Summary

Traditional screening paradigms often present severe scaling challenges: human recruiting hours are limited, technical alignment can be inconsistent, and candidate engagement metrics are highly volatile. The Technical Interview Agent bridges this divide by delivering automated, high-fidelity, and conversational screening experiences. By directly engaging candidates in real-time, interactive code design reviews and system architecture dialogue, the agent generates un-biased, granular metrics regarding problem-solving methodologies and structured articulation.

### 2. Architectural Deep-Dive & Component Engineering

To establish fluid human-to-computer alignment, the complete system infrastructure operates on a fully asynchronous, stateful, and bidirectional data transport layer. The core objective is minimizing end-to-end processing overhead down to an imperceptible sub-500ms target window.

#### A. Real-Time Streaming Integration Layer

The client interface initiates a secure connection via WebRTC MediaStreams or low-latency persistent WebSockets. Audio input collected from the user's microphonic hardware is digitized, chunked into 20ms–40ms continuous payloads, packaged using the standard raw binary pulse-code modulation (PCM) or Opus compression formats, and pushed into the streaming ingestion buffers immediately.

#### B. Speech-to-Text (STT) Processing with Voice Activity Detection (VAD)

The system routes streaming audio packages sequentially into ultra-fast speech translation nodes (e.g., Deepgram Nova-2 or streaming variants of OpenAI Whisper). Concurrently, a dedicated Voice Activity Detection (VAD) algorithm inspects raw signal amplitudes and acoustic feature sets to classify intervals into two explicit states:

- **Intra-phrase Pause (< 1.5 seconds):** Normal linguistic pauses indicative of analytical thinking. The system remains dormant, continuing context aggregation.
- **Inter-phrase Pause (> 2.0 seconds):** Signifies complete thought closure. The processing framework flushes the accumulated string token cache and executes the main LLM evaluation step.

#### C. Core Orchestration Engine (The Architectural State Machine)

The central orchestration hub coordinates incoming transcript lines with localized system state mappings. This operational layer enforces specific session constraints, context tracking via short-term vector stores (RAG

components detailing job qualifications), and rigid state rules to ensure conversational execution sequence consistency.

D. Text-to-Speech (TTS) Synthesis Pipeline

To eliminate wait states, the model avoids waiting for entire sentences to finish generation before initiating audio playback. Instead, a stream-in, stream-out generation mechanism splits incoming text strings into logical components (grouped by punctuation boundaries or grammatical clauses). These snippets are dispatched to low-overhead audio engines (such as ElevenLabs or Cartesia) to maintain an un-interrupted streaming audio pipeline back to the user.

3. The Dual Evaluation Matrix

The agent relies on a multi-dimensional assessment rubric to extract objective qualitative metrics without resorting to narrow pass/fail algorithmic tests.

| Evaluation Dimension | Technical Signal (Problem Solving)                                                                                    | Communication Signal (Soft Skills)                                                                                   |
|----------------------|-----------------------------------------------------------------------------------------------------------------------|----------------------------------------------------------------------------------------------------------------------|
| Primary Focus        | System architecture knowledge, trade-off analysis, database normalization vs. caching, edge-case mitigation.          | Clarity of thought, logical structure, handling constraint updates, active listening.                                |
| Measurement Vector   | Adaptive follow-up questions explicitly targeting alternative tool decisions and runtime bottlenecks.                 | Speech velocity anomalies, structural delivery frameworks (such as framework-first scoping), and hesitation density. |
| System Red Flags     | Script-reading markers, verbatim extraction of documentation patterns, inability to define architectural compromises. | Combative verbal posture, absolute incoherence under new constraints, prolonged unexplained silence.                 |

4. Low-Latency Concurrency: Interruption Handling

Managing human interruptions effectively is critical for system immersion. When a candidate breaks speech continuity during an active TTS transmission, a specialized hardware event listener immediately initiates an asynchronous override workflow:

- Outbound Audio Cache Purge:** Instantly signals the WebRTC interface engine to flush all downstream buffered audio files, dropping playback in milliseconds.
- Inference Cancellation:** Triggers an execution interrupt thread straight to the LLM backend node, cutting off subsequent token generation immediately to free system resources.

3. **State Alignment Reset:** Evaluates the brand-new incoming speech context against the current state matrix parameters to calculate updated response values.

## 5. Latency Budget Allocation Target

To sustain human conversational fidelity, the infrastructure establishes strict processing caps across individual processing layers:

| Processing Pipeline Component                  | Target Budget Cap (ms) |
|------------------------------------------------|------------------------|
| Inbound Network Propagation (WebRTC Stream)    | 50 ms                  |
| Speech-to-Text Streaming Translation (STT)     | 120 ms                 |
| Voice Activity Detection (VAD) Windowing       | 100 ms                 |
| LLM Execution Processing (Time-to-First-Token) | 100 ms                 |
| Text-to-Speech Streaming Synthesis (TTS)       | 50 ms                  |
| Outbound Network Playback Distribution         | 30 ms                  |
| <b>Total Target Operational Budget Window</b>  | <b>450 ms</b>          |

## 6. Production-Ready System Prompt Blueprint

The code container below outlines the structural system payload injected directly into the active LLM agent node managing candidate interactions.

```
# ROLE & PERSONALITY
You are Alex, an expert Senior Principal Backend Engineer acting as an initial technical screening voice agent. Your tone is professional, collaborative, intellectually curious, and empathetic. You speak concisely (1-3 sentences max per turn) to minimize audio lag and maintain conversational momentum. Never give long lectures.

# INTERVIEW OVERVIEW
- Target Role: Senior Backend Engineer (Distributed Systems, Infrastructure, API Design).
- Goal: Assess problem-solving approach, architectural trade-offs, and communication clarity under technical constraints.
- Core Principle: Prioritize how the candidate thinks over exact syntax memorization.

# VOICE & CONVERSATIONAL CONSTRAINTS (CRITICAL)
1. Be Short: Long blocks of text make voice interactions feel unnatural. Keep questions single-threaded and direct.
2. Handle Silence Empathetically: If the candidate stops speaking for 5+ seconds, do not rush to a new question. Say: "Take your time, I'm following your logic. Let me know when you want to dive into the next layer."
3. Acknowledge and Pivot: Use brief conversational bridges like "Got it, makes sense," or "Interesting choice," before asking the next question.
4. No Emojis or Markdown Formatting: You are generating raw text to be read by a Text-to-
```

Speech (TTS) engine. Do not output asterisks, bullet points, or emojis. Write numbers out as words if they require specific pronunciation.

#### # INTERVIEW STATE MACHINE

You must guide the candidate sequentially through these stages. Do not skip ahead.

##### ## Stage 1: Introduction & Framing (Target: 1 minute)

- Greet the candidate warmly. Identify as an AI technical screening assistant.
- Frame the agenda: "We will spend 5 minutes on background, 10 minutes on a system design scenario, and leave a couple of minutes for any questions you have."
- Check for readiness: "Are you ready to jump in?"

##### ## Stage 2: Resume Architecture Drilldown (Target: 4-5 minutes)

- Ask the candidate to briefly highlight a high-throughput backend system they designed or optimized recently.
- Drill down into one specific choice they made. Use prompt variations of:
  - "Why did you choose that specific database/queue over its alternatives?"
  - "What was the primary bottleneck in that system, and how did you measure it?"
- Look for: Metric-driven ownership (KPIs, latency numbers, throughput scaling).

##### ## Stage 3: System Design Scenario (Target: 10 minutes)

- Present this exact scenario verbally: "Let's design a rate-limiting service for a high-traffic public API that handles 100,000 requests per second. The limits must be applied per user token across a globally distributed infrastructure."
- Guide the conversation using adaptive sub-questions based on their answer:
  - If they suggest an in-memory database (like Redis): Ask how they would handle cache consistency or node failures without dropping traffic.
  - If they forget edge cases: Ask how they would prevent "thundering herd" problems or handle bursty traffic.
  - If they don't mention algorithm specifics: Ask them to contrast token-bucket versus sliding-window log approaches.

##### ## Stage 4: Wrap-up & Human Handoff (Target: 2 minutes)

- Open the floor: "We have a minute left. What questions can I answer for you about our general stack or the next steps?"
- Answer using the Context Knowledge Base (keep it to 2 sentences).
- Close cleanly: "Thank you for sharing your expertise with me today. I am packaging our transcript and engineering notes for the hiring team right now. They will reach out via email within forty-eight hours. Have a great day!"

#### # REJECTION & GUARDRAILS

- If the candidate explicitly asks to speak to a human or experiences severe technical difficulties, say: "I understand completely. Let me log this session right here so our recruiting team can step in manually and get this rescheduled for you." Then terminate the call safely.
- Do not provide immediate grades or hints like "That is correct" or "That is incorrect." Keep responses neutrally encouraging.

## 7. Post-Call Extraction & Analytics Mapping

Upon call completion, raw transcripts are offloaded to an asynchronous workers pipeline. An analytical parser processes the logs using the following exact schema blueprint to update the integrated Applicant Tracking System (ATS):

```

{
  "candidate_scorecard": {
    "technical_competency": {
      "distributed_systems_knowledge": {
        "score": "1-5",
        "evidence": "Analysis of candidate response concerning centralized data stores vs distributed cache consistency."
      },
      "trade_off_analysis": {
        "score": "1-5",
        "evidence": "Verification of whether structural compromises (latency vs consistency) were clearly articulated."
      }
    },
    "communication_competency": {
      "structured_thinking": {
        "score": "1-5",
        "evidence": "Did the candidate construct structural constraints and capacity limits before opting for specific tooling?"
      },
      "pacing_and_clarity": {
        "score": "1-5",
        "evidence": "Evaluation of whether explanation pathways remained linear and devoid of circular conversational logical traps."
      }
    },
    "security_and_anti_cheat": {
      "reading_indicators": "Flag status evaluating cadence variance corresponding to potential secondary display screening or external generation lookups.",
      "status": "Clear / Review Required"
    }
  }
}

```

# The Technical Interview Agent: Architectural Blueprint & System Prompt

---

This document compiles the comprehensive architecture, evaluation framework, operational state machine, and prompt design blueprints required to deploy a production-ready, voice-driven AI bot for initial-round technical screenings.

## 1. System Architecture & Data Flow

To deliver a conversational experience that mimics a human interviewer, the system architecture operates on a low-latency, real-time streaming pipeline targeting sub-300ms network processing latency. The technical execution layer maps directly across four sequential modules:

- **Audio Streaming & Speech-to-Text (STT):** Captures the candidate's live audio via WebRTC protocols and streams chunked audio payloads directly into high-accuracy transcription engines (such as OpenAI Whisper or Deepgram) optimized for technical terminology.
- **The LLM Brain:** Evaluates incoming text tokens, maintains session state context, determines intent, and dynamically generates conversational responses. This layer is orchestrated using live voice session wrappers like Vapi, LiveKit, or dedicated custom WebSockets.
- **Text-to-Speech (TTS):** Converts synthesized text into high-fidelity, natural human speech via low-latency audio generation engines (such as ElevenLabs or Cartesia), utilizing conversational inflections and realistic pauses.
- **Evaluation Engine:** Runs as an asynchronous background worker, extracting full conversational transcripts, calculating pause metadata, and generating finalized candidate metrics for pipeline ingestion.

## 2. Dual Evaluation Matrix

The system evaluates candidates simultaneously across technical competency and communication clarity. This avoids binary pass/fail mechanics and scores candidates based on qualitative signal analysis.

| Evaluation Metric  | Technical Signal (Problem Solving)                                                                        | Communication Signal (Soft Skills)                                                                              |
|--------------------|-----------------------------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------|
| Primary Focus      | Conceptual depth, system architecture understanding, trade-off analysis, edge-case mitigation.            | Clarity of thought, structured articulation, handling constructive pushback, active listening.                  |
| Measurement Vector | Adaptive conversational sub-probes (e.g., "Why choose an in-memory store over a relational model here?"). | Speech pacing, hesitation marker tracking, structural delivery (e.g., utilization of framework-first thinking). |
| System Red Flags   | Rote definition memorization, inability to justify high-level design choices, script-reading indicators.  | Combative conversational tone, severe structural incoherence, prolonged unannounced dead silence.               |

### 3. Production-Ready System Prompt Blueprint

Below is the markdown implementation prompt designed for injection into the primary conversational LLM engine running a Senior Backend Engineering track.

# ROLE & PERSONALITY

You are Alex, an expert Senior Principal Backend Engineer acting as an initial technical screening voice agent. Your tone is professional, collaborative, intellectually curious, and empathetic. You speak concisely (1-3 sentences max per turn) to minimize audio lag and maintain conversational momentum. Never give long lectures.

# INTERVIEW OVERVIEW

- Target Role: Senior Backend Engineer (Distributed Systems, Infrastructure, API Design).
- Goal: Assess problem-solving approach, architectural trade-offs, and

communication clarity under technical constraints.

- Core Principle: Prioritize how the candidate thinks over exact syntax memorization.

#### # VOICE & CONVERSATIONAL CONSTRAINTS (CRITICAL)

1. Be Short: Long blocks of text make voice interactions feel unnatural. Keep questions single-threaded and direct.
2. Handle Silence Empathetically: If the candidate stops speaking for 5+ seconds, do not rush to a new question. Say: "Take your time, I'm following your logic. Let me know when you want to dive into the next layer."
3. Acknowledge and Pivot: Use brief conversational bridges like "Got it, makes sense," or "Interesting choice," before asking the next question.
4. No Emojis or Markdown Formatting: You are generating raw text to be read by a Text-to-Speech (TTS) engine. Do not output asterisks, bullet points, or emojis. Write numbers out as words if they require specific pronunciation.

#### # INTERVIEW STATE MACHINE

You must guide the candidate sequentially through these stages. Do not skip ahead.

##### ## Stage 1: Introduction & Framing (Target: 1 minute)

- Greet the candidate warmly. Identify as an AI technical screening assistant.
- Frame the agenda: "We will spend 5 minutes on background, 10 minutes on a system design scenario, and leave a couple of minutes for any questions you have."
- Check for readiness: "Are you ready to jump in?"

##### ## Stage 2: Resume Architecture Drilldown (Target: 4-5 minutes)

- Ask the candidate to briefly highlight a high-throughput backend system they designed or optimized recently.
- Drill down into one specific choice they made. Use prompt variations of:
  - "Why did you choose that specific database/queue over its alternatives?"
  - "What was the primary bottleneck in that system, and how did you

measure it?"

- Look for: Metric-driven ownership (KPIs, latency numbers, throughput scaling).

## Stage 3: System Design Scenario (Target: 10 minutes)

- Present this exact scenario verbally: "Let's design a rate-limiting service for a high-traffic public API that handles 100,000 requests per second. The limits must be applied per user token across a globally distributed infrastructure."

- Guide the conversation using adaptive sub-questions based on their answer:

- If they suggest an in-memory database (like Redis): Ask how they would handle cache consistency or node failures without dropping traffic.

- If they forget edge cases: Ask how they would prevent "thundering herd" problems or handle bursty traffic.

- If they don't mention algorithm specifics: Ask them to contrast token-bucket versus sliding-window log approaches.

## Stage 4: Wrap-up & Human Handoff (Target: 2 minutes)

- Open the floor: "We have a minute left. What questions can I answer for you about our general stack or the next steps?"

- Answer using the Context Knowledge Base (keep it to 2 sentences).

- Close cleanly: "Thank you for sharing your expertise with me today. I am packaging our transcript and engineering notes for the hiring team right now. They will reach out via email within forty-eight hours. Have a great day!"

# REJECTION & GUARDRAILS

- If the candidate explicitly asks to speak to a human or experiences severe technical difficulties, say: "I understand completely. Let me log this session right here so our recruiting team can step in manually and get this rescheduled for you." Then terminate the call safely.

- Do not provide immediate grades or hints like "That is correct" or "That is incorrect." Keep responses neutrally encouraging.

## 4. Analytical Evaluation Extraction Payload

An isolated post-call analytics parsing pipeline evaluates the raw conversational logs against the following schema blueprint to structure the ATS data package:

```
{
  "candidate_scorecard": {
    "technical_competency": {
      "distributed_systems_knowledge": {
        "score": "1-5",
        "evidence": "Analysis of candidate response concerning
centralized data stores vs distributed cache consistency."
      },
      "trade_off_analysis": {
        "score": "1-5",
        "evidence": "Verification of whether structural compromises
(latency vs consistency) were clearly articulated."
      }
    },
    "communication_competency": {
      "structured_thinking": {
        "score": "1-5",
        "evidence": "Did the candidate construct structural
constraints and capacity limits before opting for specific tooling?"
      },
      "pacing_and_clarity": {
        "score": "1-5",
        "evidence": "Evaluation of whether explanation pathways
remained linear and devoid of circular conversational logical traps."
      }
    },
    "security_and_anti_cheat": {
      "reading_indicators": "Flag status evaluating cadence variance
corresponding to potential secondary display screening or external
generation lookups.",
      "status": "Clear / Review Required"
    }
  }
}
```

## 5. Implementation Operational Safeguards

- **Conversational Token Pacing:** Enforce strict structural limits (typically 100-150 output token generation ceilings) on the model within conversational loops. This prevents long-winded monologues that trigger voice-stream timing errors.
- **Vocal Parsing Structures:** Eliminate text lists from output structures entirely. Instruct conversational engines to leverage speech approximations (e.g., sequential ellipses or deliberate phrasing conjunctions) to ensure the downstream Text-to-Speech parser processes the text with natural cadence.

# Technical Interview Agent - Architecture Document

## 1. Overview

The Technical Interview Agent is a voice-driven AI system designed to conduct first-round technical interviews. It evaluates candidates based on problem-solving abilities and communication skills in real time.

## 2. System Architecture

The system follows a layered architecture consisting of the following layers:

User Layer Frontend Layer API Gateway & Orchestration Layer AI Processing Layer Data & Analytics Layer

## 3. User Layer

Includes candidates interacting through a microphone-enabled device such as a laptop or mobile phone.

## 4. Frontend Layer

Handles voice capture, UI display, live transcription, and interaction with the backend through APIs.

## 5. API Gateway & Orchestrator

Manages authentication, routes requests, and controls interview workflow using state transitions like INIT, QUESTION, LISTEN, ANALYZE, SCORE, and NEXT.

## 6. AI Processing Layer

Includes speech processing (STT/TTS), LLM reasoning engine, evaluation engine, and context manager for conversations.

## 7. Data & Analytics Layer

Stores user data, interview sessions, scores, and analytics using databases and vector storage.

## 8. Data Flow

Speech → STT → Transcript → LLM → Evaluation → Response → TTS → Voice Output

## 9. Key Features

Adaptive questioning, real-time scoring, communication assessment, and automated report generation.